

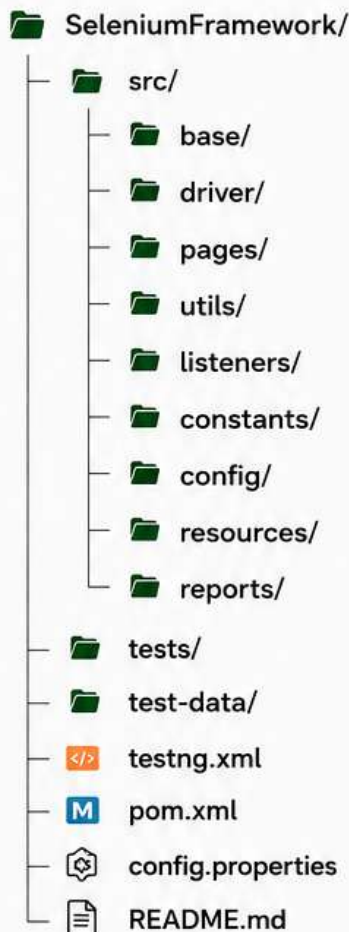
WHAT IS AN AUTOMATION FRAMEWORK?

An Automation Framework is a set of guidelines, best practices, and reusable components that help us build scalable, maintainable and reliable test automation solutions.

HIGH LEVEL ARCHITECTURE



PROJECT STRUCTURE



★ KEY BENEFITS

- ✓ Scalable & Maintainable
- ✓ Reusable Components
- ✓ Easy to Integrate with CI/CD
- ✓ Parallel Execution Support
- ✓ Better Reporting & Logging
- ✓ Environment & Data Management
- ✓ Cross Browser Testing
- ✓ Data Driven Testing
- ✓ Retry Mechanism for Failures

DESCRIPTION OF EACH COMPONENT

- **src/**
Contains all the source code of the framework.
- **base/**
Holds base classes like BaseTest, BasePage with common setup, teardown and reusable methods.
- **driver/**
Manages WebDriver instances and driver factory classes for browser initialization.
- **pages/**
Page Object classes. Each page of the application will have a separate class with locators and actions.
- **utils/**
Utility classes with reusable methods like WaitUtils, ScreenshotUtils, DateUtils, ExcelUtils, etc.
- **listeners/**
TestNG Listeners for reporting, logs, screenshots on failure, retry analyzer, etc.
- **constants/**
Holds constant values like timeouts, messages, file paths, keys, etc.
- **config/**
Configuration related classes to read properties from config files (env, urls, credentials, etc).
- **resources/**
Contains external resources like properties files, test data, xml files, templates, etc.
- **reports/**
Generated reports will be stored here (Extent Report, logs, screenshots).
- **tests/**
Contains all the test classes written using TestNG.
- **test-data/**
Stores test data files like excel, csv, json, yaml, etc.
-  **testng.xml** TestNG suite file. Used to group and run test classes.
-  **pom.xml** Maven project object model. Manages dependencies and build configurations.
-  **config.properties** Framework configuration file. Defines browser, url, timeout, etc.
-  **README.md** Project documentation and setup instructions.

i FRAMEWORK HIGHLIGHTS



Page Object Model (POM)



Data Driven Testing



Parallel Execution



Cross Browser Support



Logging & Monitoring



Reporting & Screenshots



Retry Failed Tests

BASE CLASSES & DRIVER MANAGEMENT

These classes handle test setup, teardown and browser initialization.

1

BaseTest



Base class for all test classes.
Handles setup, teardown, screenshots and reporting.

BaseTest.java

```

1  package base;
2  import org.openqa.selenium.WebDriver;
3  import org.testng.ITestResult;
4  import org.testng.annotations.*;
5  import utils.ScreenshotUtils;
6  import driver.DriverFactory;
7  import reports.ExtentReportManager;
8
9  public class BaseTest {
10     protected WebDriver driver;
11
12     @BeforeMethod(alwaysRun = true)
13     public void setup(ITestResult result) {
14         driver = DriverFactory.getDriver();
15         ExtentReportManager.startTest(result.getMethod().getMethodName());
16     }
17
18     @AfterMethod(alwaysRun = true)
19     public void tearDown(ITestResult result) {
20         if(ITestResult.FAILURE == result.getStatus()) {
21             ScreenshotUtils.captureScreenshot(driver, result.getName());
22             ExtentReportManager.addScreenCapture(result.getName());
23         }
24         DriverFactory.quitDriver();
25         ExtentReportManager.endTest(result.getStatus());
26     }
27 }

```

2

DriverFactory



Factory class to create and manage WebDriver instances.
Uses ThreadLocal for parallel execution.

DriverFactory.java

```

1  package driver;
2  import java.time.Duration;
3  import org.openqa.selenium.WebDriver;
4  import org.openqa.selenium.chrome.ChromeDriver;
5  import org.openqa.selenium.edge.EdgeDriver;
6  import org.openqa.selenium.firefox.FirefoxDriver;
7  import utils.ConfigReader;
8
9  public class DriverFactory {
10     private static ThreadLocal<WebDriver> driverThread = new ThreadLocal<>();
11
12     public static WebDriver getDriver() {
13         if(driverThread.get() != null) {
14             return driverThread.get();
15         }
16         String browser = ConfigReader.getProperty("browser").toLowerCase();
17         WebDriver driver;
18         switch (browser) {
19             case "chrome": driver = new ChromeDriver(); break;
20             case "firefox": driver = new FirefoxDriver(); break;
21             case "edge": driver = new EdgeDriver(); break;
22             default:
23                 throw new RuntimeException("Browser not supported: " + browser);
24         }
25         driver.manage().window().maximize();
26         driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
27         driverThread.set(driver);
28         return driver;
29     }
30     public static void quitDriver() {
31         WebDriver driver = driverThread.get();
32         if(driver != null) {
33             driver.quit();
34             driverThread.remove();
35         }
36     }
37 }

```

3

BasePage



Parent class for all page classes.
Contains common methods for web actions.

BasePage.java

```

1  package base;
2  import java.time.Duration;
3  import org.openqa.selenium.*;
4  import org.openqa.selenium.support.ui.ExpectedConditions;
5  import org.openqa.selenium.support.ui.WebDriverWait;
6
7  public class BasePage {
8     protected WebDriver driver;
9     protected WebDriverWait wait;
10
11     public BasePage(WebDriver driver) {
12         this.driver = driver;
13         this.wait = new WebDriverWait(driver, Duration.ofSeconds(30));
14     }
15
16     public void click(By locator) { wait.until(ExpectedConditions.elementToBeClickable(locator)).click(); }
17     public void type(By locator, String text) {
18         wait.until(ExpectedConditions.visibilityOfElementLocated(locator)).clear();
19         driver.findElement(locator).sendKeys(text);
20     }
21 }

```


PAGE OBJECTS, CONFIG & UTILITIES

Reusable components that make the framework flexible, data-driven and maintainable.

1

LoginPage



Page class for Login functionality. Contains locators and actions.

LoginPage.java

```
1 package pages;
2 import org.openqa.selenium.By;
3 import org.openqa.selenium.WebDriver;
4 import base.BasePage;
5
6 public class LoginPage extends BasePage {
7     private By username = By.id("username");
8     private By password = By.id("password");
9     private By loginBtn = By.id("loginBtn");
10
11     public LoginPage(WebDriver driver) {
12         super(driver);
13     }
14
15     public HomePage login(String user, String pass) {
16         type(username, user);
17         type(password, pass);
18         click(loginBtn);
19         return new HomePage(driver);
20     }
21 }
```

2

HomePage



Page class for Home page actions and validations.

HomePage.java

```
1 package pages;
2 import org.openqa.selenium.By;
3 import org.openqa.selenium.WebDriver;
4 import base.BasePage;
5
6 public class HomePage extends BasePage {
7     private By logoutBtn = By.id("logout");
8
9     public HomePage(WebDriver driver) {
10         super(driver);
11     }
12     public LoginPage logout() {
13         click(logoutBtn);
14         return new LoginPage(driver);
15     }
16 }
```

3

ConfigReader



Reads configuration from config.properties file.

ConfigReader.java

```
1 package config;
2 import java.io.FileInputStream;
3 import java.io.IOException;
4 import java.util.Properties;
5
6 public class ConfigReader {
7     private static Properties prop;
8     static {
9         prop = new Properties();
10        try (FileInputStream fis = new FileInputStream("config.properties")) {
11            prop.load(fis);
12        } catch (IOException e) {
13            e.printStackTrace();
14        }
15    }
16    public static String getProperty(String key) {
17        return prop.getProperty(key);
18    }
19 }
```

4

WaitUtils



Explicit wait helper methods to handle synchronization.

WaitUtils.java

```
1 package utils;
2 import org.openqa.selenium.*;
3 import org.openqa.selenium.support.ui.*;
4 import java.time.Duration;
5
6 public class WaitUtils {
7     public static WebDriverWait getWait(WebDriver driver, int timeout) {
8         return new WebDriverWait(driver, Duration.ofSeconds(timeout));
9     }
10
11     public static WebElement waitForVisibility(WebDriver driver, By locator, int timeout) {
12         return getWait(driver, timeout).until(ExpectedConditions.visibilityOfElementLocated(locator));
13     }
14     public static WebElement waitForClickable(WebDriver driver, By locator, int timeout) {
15         return getWait(driver, timeout).until(ExpectedConditions.elementToBeClickable(locator));
16     }
17 }
```

5

ScreenshotUtils



Utility class to capture screenshots.

ScreenshotUtils.java

```
1 package utils;
2 import org.openqa.selenium.OutputType;
3 import org.openqa.selenium.TakesScreenshot;
4 import org.openqa.selenium.WebDriver;
5 import java.io.File;
6 import java.io.IOException;
7 import org.apache.commons.io.FileUtils;
8
9 public class ScreenshotUtils {
10     public static String captureScreenshot(WebDriver driver, String testName) throws IOException {
11         TakesScreenshot ts = (TakesScreenshot) driver;
12         File src = ts.getScreenshotAs(OutputType.FILE);
13         String filePath = System.getProperty("user.dir") + "/reports/screenshots/" + testName + "_" + System.currentTimeMillis() + ".png";
14         FileUtils.copyFile(src, new File(filePath));
15         return filePath;
16     }
17 }
```

UTILITIES, REPORTING & LISTENERS

These components handle logging, reporting, retry logic and test execution events.

1

LoggerUtils



Utility class to create and manage logs using Log4j/SLF4J.

LoggerUtils.java

```
1 package utils;
2 import org.apache.logging.log4j.LogManager;
3 import org.apache.logging.log4j.Logger;
4
5 public class LoggerUtils {
6     private static final Logger log = LogManager.getLogger(LoggerUtils.class);
7
8     public static void info(String message) {
9         log.info(message);
10    }
11
12    public static void warn(String message) {
13        log.warn(message);
14    }
15
16    public static void error(String message) { log.error(message); }
17 }
```

2

ExcelUtils



Utility class to read/write data from/to Excel files using Apache POI.

ExcelUtils.java

```
1 package utils;
2 import java.io.FileInputStream;
3 import org.apache.poi.ss.usermodel.*;
4 import org.apache.poi.xssf.usermodel.XSSFWorkbook;
5
6 public class ExcelUtils {
7     public static String getCellData(String filePath, String sheetName,
8                                     int rowNum, int colNum) throws Exception {
9         FileInputStream fis = new FileInputStream(filePath);
10        Workbook workbook = new XSSFWorkbook(fis);
11        Sheet sheet = workbook.getSheet(sheetName);
12        return sheet.getRow(rowNum).getCell(colNum).toString();
13    }
14 }
```

3

RetryAnalyzer



Retry failed tests based on max retry count.

RetryAnalyzer.java

```
1 package listeners;
2 import org.testng.IRetryAnalyzer;
3 import org.testng.ITestResult;
4
5 public class RetryAnalyzer implements IRetryAnalyzer {
6     private int retryCount = 0;
7     private int maxRetryCount = 2;
8
9     @Override
10    public boolean retry(ITestResult result) {
11        if (retryCount < maxRetryCount) {
12            retryCount++;
13            return true;
14        }
15        return false;
16    }
17 }
```

4

TestListener



TestNG Listener to capture test execution events.

TestListener.java

```
1 package listeners;
2 import org.testng.*;
3 import utils.LoggerUtils;
4
5 public class TestListener implements ITestListener {
6     @Override
7     public void onTestStart(ITestResult result) {
8         LoggerUtils.info("Test Started: " + result.getName());
9     }
10
11    @Override
12    public void onTestSuccess(ITestResult result) {
13        LoggerUtils.info("Test Passed: " + result.getName());
14    }
15
16    @Override
17    public void onTestFailure(ITestResult result) {
18        LoggerUtils.error("Test Failed: " + result.getName());
19    }
20 }
```

5

ExtentReportManager



Manages Extent Report creation and logging.

ExtentReportManager.java

```
1 package reports;
2 import com.aventstack.extentreports.*;
3
4 public class ExtentReportManager {
5     private static ExtentReports extent;
6     private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();
7
8     public static ExtentReports getInstance() {
9         if (extent == null) {
10            extent = new ExtentReports("reports/ExtentReport.html", true);
11        }
12        return extent;
13    }
14 }
```


This page covers test class example, TestNG configuration, Maven commands and CI/CD flow.

1

Sample Test Class



- ✓ Uses Page Objects
- ✓ Data Driven Ready
- ✓ Clean & Readable
- ✓ Easy to Maintain

LoginTest.java

```
1 package tests;
2 import org.testng.annotations.*;
3 import pages.LoginPage;
4 import pages.HomePage;
5 import base.BaseTest;
6
7 public class LoginTest extends BaseTest {
8     LoginPage loginPage;
9     HomePage homePage;
10
11     @Test(dataProvider = "loginData", retryAnalyzer = listeners.RetryAnalyzer.class)
12     public void loginTest(String user, String pass) {
13         loginPage = new LoginPage(driver);
14         homePage = loginPage.login(user, pass);
15         Assert.assertTrue(homePage.isHomePageDisplayed(), "Login Failed");
16         homePage.logout();
17     }
18
19     @DataProvider(name = "loginData")
20     public Object[][] getData() {
21         return new Object[][] { { "admin", "admin123"}, {"user", "user123"} }; }
22 }
```

2

testng.xml



TestNG suite configuration to group and run test cases.

testng.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
3 <suite name="SeleniumFrameworkSuite" parallel="tests" thread-count="3">
4     <listeners>
5         <listener class-name="listeners.TestListener"/>
6         <listener class-name="org.uncommons.reporting.HTMLReporter"/>
7     </listeners>
8     <test name="LoginTests">
9         <classes>
10             <class name="tests.LoginTest"/>
11         </classes>
12     </test>
13 </suite>
```

3

Maven Commands



```
# Clean and run all tests
mvn clean test # Runs the tests and generates reports

# Run tests with specific suite
mvn test -DsuiteXmlFile=testng.xml # Executes tests from testng.xml

# Run tests on a specific browser
mvn test -Dbrowser=chrome # Passes browser as system property

# Generate report only
mvn test -DgenerateReports=true # Generates Extent report only
```

4

CI/CD Flow (Jenkins)



5

Framework Features



- ✓ **Page Object Model (POM)**
Each page has a separate class with locators and actions.
- ✓ **Data Driven Testing**
Supports Excel / CSV / JSON for data-driven test execution.
- ✓ **Parallel Execution**
Run tests in parallel across browsers and threads using TestNG.
- ✓ **Cross Browser Testing**
Supports Chrome, Firefox, Edge and more browsers.
- ✓ **Reusable Components**
Utilities, base classes and helpers make tests reusable.
- ✓ **Explicit Waits**
Robust wait methods using WebDriverWait for stability.
- ✓ **Retry Mechanism**
Automatically retry failed tests using RetryAnalyzer.
- ✓ **Logging**
Logs at each step using Log4j for better debugging.
- ✓ **Reporting**
Detailed Extent Reports with steps, logs and screenshots.
- ✓ **Screenshot on Failure**
Captures screenshots automatically on test failure.
- ✓ **CI/CD Ready**
Easily integrate with Jenkins for continuous testing.
- ✓ **Maintainable & Scalable**
Clean structure, modular design and easy to extend.

COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.